

Swift: 基于强化学习的多信号融合自适应网络拥塞控制算法

杭天铨

南京邮电大学

1224045520@njupt.edu.cn

摘要

端到端网络传输的服务对象正在从固定终端和近端高质量链路, 扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入路径, 以及手机、笔记本、台式机和边缘设备等异构终端并存的复杂场景。传统基于丢包的算法存在拥塞信号滞后与缓冲区膨胀问题, 基于延迟的算法面临 RTT 测量噪声与共存公平性不足等缺陷; 当长 RTT、链路抖动与终端能力差异同时出现时, 固定参数、单一信号驱动的拥塞控制更难兼顾吞吐、延迟与稳定性。本文提出 Swift——一种面向远距离传输与异构终端的强化学习辅助多信号融合自适应拥塞控制方法, 其核心创新收敛为三点: (1) **多信号拥塞状态感知**, 将 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间 (嵌入 15 元素 OpenGym 传输容器, 另含套接字标识、环境类型、仿真时间、节点标识四项跨进程路由元数据), 并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包; (2) **强化学习辅助的自适应融合控制**, 由最小 RTT 感知的 RTT 膨胀比、环境奖励相对其自身慢速基线的偏移量以及连续增长趋势共同调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$, 并以有界步长逼近 $\alpha \times BDP$ 目标窗口, 其中 BDP 由滑动时间窗口投递速率的窗口最大值滤波给出; (3) **面向不确定拥塞信号的稳定性与安全机制**, 包括差异化窗口保留因子 (丢包 $\rho = 0.70$ 、ECN $\rho = 0.75$ 、超时 $\rho = 0.50$)、连续递减下限 $D_{\max} = 3$ 、降窗后 4 个 ACK 事件冻结、不奖励队列驻留的慢启动阈值更新以及窗口安全钳位。

在 ns-3.40 离散事件仿真平台上, 本文以哑铃拓扑、3 条并发长流以及 800 Mbps UDP 突发流对照设置评估 Swift。为消除既有汇总表将反向 ACK 流计入指标的缺陷 (该缺陷使数据面延迟被折半、吞吐量被抬高约 1.7%), 本文仅按前向数据流重新导出全部 KPI (logs/summary/kpi_forward.csv): 经四条排除规则审计后, 纯 TCP 设置保留 19 个场景、UDP 突发设置保留 15 个场景。结果表明, 在微秒级 RTT 高速链路上 Swift 与 NewReno/CUBIC 的吞吐量在三位小数上完全一致 (差异 +0.000%), 而平均单向延迟降低 82.1%~93.6%; 在 5G eMBB、802.11ax/ac 与城域 WAN 等中速接入场景延迟降低 10.2%~41.0%; 在 GEO 卫星链路上相对 NewReno/CUBIC 均值提升吞吐量 29.8%, UDP 突发下该优势扩大至 87.8%; Swift 在 19/19 (纯 TCP) 与 15/15 (UDP 突发) 个场景中丢包率均为 0.00%, Jain 公平性指数均值 1.000, UDP 突发下吞吐量保持率 80.54%, 优于 NewReno (75.86%)、CUBIC (78.78%) 与 BBR (39.07%)。本文同时如实报告混合与负向结果: 跨域广域网场景 (cross_dc_wan) 吞吐量较 CUBIC 低约 29%, 其机理为修复前的逐 ACK 投递速率估计系统性低估带宽, v3.0.0 已替换为时间窗口估计; 在低速与长 RTT 接入链路上, Swift 以队列驻留换取零丢包, 平均单向延迟相对更优的丢包类基线抬升 7%~129%。全部定量结果均来自 v3.0.0 修复前的实验产物, 全矩阵重跑尚未完成, 故上述数值构成本文的证据边界。

关键词: 拥塞控制; 强化学习; 多信号融合; 显式拥塞通知;

自适应参数调优; 远距离传输; 异构终端

1 引言

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展, 端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等异构终端, 传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。TCP 作为互联网传输层的核心协议, 其拥塞控制机制的性能直接影响网络资源利用效率、应用响应延迟和用户体验。

在这类端到端传输场景中, 网络路径和终端侧条件呈现出更强的异构性。一方面, 城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动; 另一方面, 手机、电脑、边缘设备等终端在处理能力、无线接入方式、移动性和业务负载上差异显著。同一拥塞控制策略需要在短 RTT 近端高速链路与长 RTT 远距离链路之间切换, 并在突发跨流量、无线误码和终端负载波动下保持稳定。上述复杂性使传统依赖固定参数或单一拥塞信号的算法难以同时满足高吞吐、低延迟和低丢包目标。

传统基于丢包的拥塞控制算法 (如 NewReno [2]、CUBIC [3]) 将丢包作为拥塞信号, 但丢包是拥塞的滞后指示器, 通常意味着网络缓冲区已经溢出, 排队延迟已累积到较高水平, 产生 "Bufferbloat" 问题 [4]。基于延迟的算法 (如 Vegas [5]、BBR [6]) 尝试通过 RTT 变化提前感知拥塞, 但 RTT 测量易受操作系统调度、路径抖动等因素干扰, 且与基于丢包算法共存时存在严重的公平性问题 [8]。

ECN 机制 [12] 允许网络设备在队列超过阈值时标记数据包而非丢弃, 实现更早、更精确的拥塞通知。DCTCP [13] 利用 ECN 标记比例进行细粒度调整, 取得了良好效果。然而, 现有算法对 ECN 信号的利用仍不够充分, 大多采用静态参数配置, 缺乏自适应能力。

近年来, 强化学习在网络拥塞控制中的应用受到关注 [15]。Aurora [17] 采用 PPO 算法训练深度神经网络策略, Orca [18] 使用强化学习动态调整 AIMD 参数。然而, 现有方案存在状态空间设计不完善 (通常仅 4-6 维)、未利用 ECN 和拥塞控制状态机信息、奖励函数难以平衡多目标、训练效率低等问题。

与单一网络域相比, 跨地域和多接入传输的拥塞信号更难解释。丢包可能来自真实队列溢出, 也可能来自无线误码或链路切换; RTT 升高既可能表示排队, 也可能来自路径变化、终端调度延迟或接入链路抖动; 突

发 UDP/RPC/视频/备份流量还会造成短时间带宽挤占。因此, Swift 的设计目标不是某一类专用网络, 而是端到端路径条件高度可变的通用传输场景。相应地, 本文将微秒级 RTT 的近端高速链路、无线局域网、蜂窝接入、城域/跨域广域网与 GEO 卫星链路组织为一组覆盖不同 RTT 量级、带宽量级和接入条件的评估集合; 其中短 RTT 高速率链路(如机架内与叶脊互联)仅作为该集合中 RTT 最小、排队延迟占比最高的一类被评估的网络条件出现, 并不构成算法的设计动机或部署前提。

针对上述挑战, 本文提出 Swift 算法——一种基于强化学习辅助的多信号融合自适应网络拥塞控制方法, 其决策策略以每个 ACK 事件为粒度在协议栈之外求解, 经 ns3-gym [22] 接口与 ns-3.40 协议栈交互。本文的核心创新收敛为如下三点:

(1) 面向远距离与异构终端的多信号拥塞状态感知: 将 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间, 并据此以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包, 为异构终端与多接入路径提供比“仅丢包”或“仅 RTT”更完整的拥塞语义。

(2) 强化学习辅助的自适应融合控制: 由最小 RTT 感知的 RTT 膨胀比、环境奖励相对其自身慢速基线的偏移量以及连续增长趋势三路信号共同调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$, 并以有界步长逼近 $\alpha \times BDP$ 目标窗口; BDP 由滑动时间窗口投递速率的窗口最大值滤波给出, 配合最小 RTT 自适应的 HyStart 风格慢启动退出与管道利用率感知加权递增, 在吞吐、延迟与稳定性之间取得在线折中。

(3) 面向不确定拥塞信号的稳定性与安全机制: 依据信号严重程度施加差异化窗口保留因子(丢包 0.70、ECN 0.75、超时 0.50), 并以连续递减下限 $D_{\max} = 3$ 、降窗后 4 个 ACK 事件冻结、不奖励队列驻留的慢启动阈值更新、窗口安全钳位以及超时丢弃陈旧动作等机制, 抑制误判信号与陈旧带宽估计导致的窗口过度收缩和振荡。

2 相关工作

2.1 传统拥塞控制算法

2.1.1 基于丢包的算法

TCP Tahoe 和 Reno 是最早的拥塞控制算法 [1], 引入了慢启动、拥塞避免和快速重传/恢复机制。TCP NewReno [2] 改进了快速恢复机制, 能够处理突发丢包情况, 是 RFC 6582 定义的标准算法。

TCP CUBIC [3] 是 Linux 自 2.6.19 版本以来的默认算法, 采用三次函数模型调整窗口:

$$W(t) = C \cdot (t - K)^3 + W_{\max} \quad (1)$$

其中 $K = \sqrt[3]{W_{\max} \cdot \beta / C}$ 。CUBIC 的窗口增长与 RTT 无关, 在高 BDP 网络中具有更快的收敛速度。

基于丢包算法的共同缺陷包括: (1) 拥塞信号滞后, 缓冲区溢出前已积累大量排队延迟; (2) 缓冲区膨

胀, 持续增加发送速率直至丢包; (3) 无法区分拥塞程度, 对所有拥塞采用统一的窗口减半策略。

2.1.2 基于延迟的算法

TCP Vegas [5] 通过比较期望吞吐量 $cwnd/BaseRTT$ 与实际吞吐量 $cwnd/RTT$ 的差值 Δ 调整窗口, 当 $\Delta < \alpha$ 时增窗, $\Delta > \beta$ 时降窗。

BBR [6] 通过估计瓶颈带宽 $BtlBw$ 和最小 RTT RTT_{prop} , 以 BDP 为目标调整发送速率: $pacing_rate = pacing_gain \times BtlBw$ 。BBR 采用周期性增益调整实现带宽探测。

Copa [7] 将目标排队延迟设为 $RTT_{standing} - RTT_{min}$ 的函数, 并引入竞争模式检测机制。

这类算法面临 RTT 测量不稳定(受调度延迟、路径抖动影响)、与丢包算法共存公平性差(研究表明 BBR 与 CUBIC 共存时吞吐量可下降 30% 以上 [8])、参数敏感等问题。

2.1.3 混合算法

Compound TCP [9] 叠加基于延迟的窗口分量, Illinois [10] 根据 RTT 调整 AIMD 参数, Westwood [11] 通过 ACK 到达率估计带宽。混合算法在一定程度上缓解了单一信号的局限性, 但增加了复杂性。

2.2 ECN 机制与相关算法

ECN [12] 通过 IP 报头中的 2 比特字段传递拥塞信息, 定义四种码点 (Non-ECT、ECT(0)、ECT(1)、CE)。发送端设置 ECT 标记表明支持 ECN; 网络设备在队列超阈值时将 ECT 改为 CE; 接收端回传 ECE 标志; 发送端响应后设置 CWR 标志。

DCTCP [13] 利用 ECN 标记比例 α 进行细粒度调整:

$$\alpha = (1 - g) \cdot \alpha + g \cdot F, \quad cwnd = cwnd \times (1 - \alpha/2) \quad (2)$$

其中 $g = 1/16$ 是平滑因子, F 是最近一个 RTT 内被标记的数据包比例。DCTCP 能根据拥塞程度进行比例响应, 但参数静态预设。HPCC [14] 利用网络设备提供的 INT 精确拥塞信息进行控制, 但需要特殊硬件支持。

ECN 的优势在于: (1) 在丢包前提供明确拥塞信号; (2) 避免重传开销; (3) 信号来自网络设备显式标记, 比 RTT 推测更可靠。然而, 现有算法对 ECN 的利用仍不充分。

2.3 基于强化学习的拥塞控制

Aurora [17] 采用 PPO 算法训练神经网络策略, 使用 10 维状态空间(延迟梯度、延迟比率、发送比率等), 奖励函数为 $r = throughput - \delta \cdot delay - \lambda \cdot loss$ 。Orca [18] 在 AIMD 框架下用强化学习动态调整增加因子 α 和减少因子 β 。PCC [20] 定义实用函数 $U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x$, 通过梯度上升最大化。Remy [16] 采用离线优化生成规则。

现有方法的共同问题是: (1) 状态空间较窄, 通常为 4~6 维, 未利用 ECN 和 CA 状态等协议栈内部信息; (2) 奖励函数设计困难, 多目标权衡复杂; (3) 训练与再训练成本高; (4) 在训练分布之外的路径条件

上泛化能力有限。Swift 在状态表示上引入协议栈内部信号，并以确定性规则代替神经网络推理，从而避免训练依赖；但本文不主张 Swift 在表示能力上优于深度策略，二者的取舍差异见第5节。

2.4 参数化白盒控制与带外参数优化

与“用学习模型替换控制逻辑”不同，另一类工作把学习限制在参数层面。Gemini [19] 采用分而治之的框架：其参数化拥塞控制类 Fusion 以 Linux 内核模块形式实现固定的白盒控制逻辑（融合窗口式与速率式增长，以丢包率阈值 l 和 RTT 膨胀阈值 δ 作为拥塞信号，并以 $\alpha \cdot R^{\max} T^{\min}$ 为窗口目标），其在线优化引擎 Booster 则运行在用户态集群中，按分钟级时间窗口聚合流级统计量，并用贝叶斯优化在隐变量（区域、ISP、时段）分组内搜索控制参数 $\theta = \{\omega, \alpha, \gamma, \lambda, l, \delta, n\}$ 。该设计的优势是数据面开销与传统算法相当，且已在生产网络完成大规模部署验证。

Swift 与之处于不同的设计点。第一，**决策位置与时间尺度**：Gemini 的控制逻辑固定在内核数据面内，学习只在带外、以分钟级和流组粒度更新参数；Swift 把整个决策策略置于协议栈之外，并在每个 ACK 事件（GetSsThresh/IncreaseWindow 回调）上被同步调用，因此其自适应发生在单流、单事件粒度上。第二，**可用信号**：带外优化器只能观察流级统计量，而 Swift 的观测向量直接来自协议栈内部，可读取 ECN 状态机与拥塞避免状态机，从而区分 ECE/CWR 触发的收缩与真实丢包。第三，**部署代价**：这一选择的代价是每事件一次跨进程往返，本文实现依赖 ZeroMQ 同步通信，在当前形态下适用于仿真与研究平台，而非生产数据面；Gemini 的内核态 Fusion 在这一维度上明显更易部署。第四，**证据强度**：Gemini 给出了生产网络 A/B 测试结果，本文的证据仅来自 ns-3.40 仿真。因此，本文将 Swift 定位为“协议栈外每事件决策 + 协议栈内信号”这一设计点的可行性与代价评估，而非对 Gemini 式部署路线的替代。

3 Swift 算法设计

3.1 系统架构

Swift 的系统架构由四个核心模块组成。**网络仿真模块**基于 ns-3 [21] 构建，实现完整 TCP/IP 协议栈和多种网络拓扑（近端高速链路、哑铃型、无线/移动、广域网与卫星链路等）。**拥塞控制环境模块**基于 OpenGym [22] 框架实现，在 TCP 协议栈的五个关键回调点（GetSsThresh、IncreaseWindow、PktsAacked、CongestionStateSet、CwndEvent）采集 11 维状态参数，并将智能体决策应用于 TCP 连接。**智能决策模块**采用 Python 实现，执行多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算，维护每个 TCP 连接的独立状态。**进程间通信模块**采用 ZeroMQ 消息队列和 Protocol Buffers 序列化，实现仿真进程与决策进程间的同步交互；由于 OpenGym 路径依赖 ZMQ 同步通信，当前 TcpSwift 实验未启用 MTP 并行仿真。

Swift 将拥塞控制建模为马尔可夫决策过程 $\langle S, A, P, R, \gamma \rangle$ 。状态空间 S 为 11 维有效 RL 观测子

空间（嵌入于 15 元素 OpenGym 传输容器，另含 4 项元数据通道），动作空间 A 直接输出新的 ssthresh 和 cwnd 值（连续动作空间），采用基于规则的确定性策略而非神经网络，以提高可解释性并避免神经网络推理依赖。

状态空间设计满足以下理论要求：（1）**马尔可夫性**——当前状态包含做出最优决策所需的全部信息，即 $P(s_{t+1}, r_{t+1} | s_t, a_t, s_{t-1}, \dots) = P(s_{t+1}, r_{t+1} | s_t, a_t)$ ，通过包含 ECN 状态、CA 状态等协议栈内部状态近似满足；（2）**信息充分性**——11 维空间涵盖 TCP 协议栈所有关键信息，避免因信息不足导致的次优决策；（3）**可观测性**——所有参数均可从 ns-3 标准回调接口直接获取；（4）**维度可控性**——11 维处理时间复杂度 $O(1)$ ，基于规则的决策避免了维度灾难问题。

3.2 11 维观测空间设计

Swift 的观测空间采用“11 维有效 RL 子空间 + 4 项元数据通道”的 15 元素 OpenGym 传输布局，如表1所示。参数涵盖六大类别：连接标识（socketUuid、nodeId）用于多流识别和状态隔离；时间信息（envType、simTime）用于时序分析和投递速率计算；窗口状态（ssthresh、cwnd）是拥塞控制的核心变量；传输性能（segSize、segmentsAacked、bytesInFlight）用于吞吐量估算和管道利用率计算；延迟测量（lastRtt、minRtt）用于 BDP 估算和拥塞程度判断；回调类型与拥塞控制状态（calledFunc、caState、caEvent、ecnState）提供协议栈完整状态。其中前 4 项（socketUuid、envType、simTime、nodeId）为跨进程路由用的元数据，不参与窗口决策；后 11 项构成参与 AIMD/ECN 响应/ α 自适应的有效 RL 状态，故本文后续统称“11 维观测”指该有效子空间。

表 1: Swift 11 维有效观测子空间（对应 15 元素容器索引 4~14）

索引	参数	说明
4	ssThresh	慢启动阈值 (B)
5	cwnd	拥塞窗口 (B)
6	segSize	段大小 (B)
7	segmentsAacked	确认段数
8	bytesInFlight	在途字节数
9	lastRtt	最近 RTT(μ s)
10	minRtt	最小 RTT(μ s)
11	calledFunc	回调类型 (0/1)
12	caState	CA 状态 (0-4)
13	caEvent	CA 事件 (0-5)
14	ecnState	ECN 状态 (0-5)

Swift 的核心创新在于引入三个协议栈内部状态参数。**caState** 反映 TCP 拥塞控制状态机的当前状态：CA_OPEN (0, 正常传输)、CA_DISORDER (1, 收到重复确认)、CA_CWR (2, 响应 ECN 缩减窗口)、CA_RECOVERY (3, 快速恢复中)、CA_LOSS (4, 超时丢包)。**caEvent** 表示最近的拥塞事件：包括 CA_EVENT_TX_START (0)、CWND_RESTART (1)、COMPLETE_CWR (2)、LOSS (3)、ECN_NO_CE (4)、ECN_IS_CE (5) 六种。**ecnState** 反映 ECN 协商和标记状态：ECN_DISABLED (0)、ECN_IDLE (1)、ECN_CE_RCVD (2)、ECN_SENDING_ECE (3)、ECN_ECE_RCVD (4)、ECN_CWR_SENT (5)。

这三个参数的引入使智能体能够感知 TCP 协议栈的完整状态信息，近似满足马尔可夫性要求。所有 15 个参数均可从 ns-3 TCP 模块的标准回调接口直接获取，满足可观性要求。11 维有效状态的处理时间复杂度为 $O(1)$ ，避免了神经网络策略的高维推理依赖。

3.3 多信号融合拥塞检测

Swift 将拥塞信号分为三类，采用两级优先级层次结构进行综合判断，并引入连续递减保护机制防止窗口过度收缩。

第一优先级——窗口收缩回调检测：当 $calledFunc = 0$ 时（GetSsThresh 回调），表明协议栈正在执行窗口收缩。此时进一步区分收缩原因：若 $caState = CA_LOSS$ ，判定为超时丢包（通常意味着连续多个数据包丢失或网络路径中断）；若 $ecnState \in \{ECN_CE_RCVD, ECN_ECE_RCVD\}$ 或 $caState = CA_CWR$ ，说明本次收缩由 ECN 回显（ECE）触发，判定为 ECN 标记并施加 ECN 保留因子；否则判定为普通丢包（快速重传触发）。丢包和超时是最可靠的拥塞信号（似然比 $LR \approx 100$ ），一旦检测到即触发响应。

第二优先级——ECN 状态直接检测：当 $ecnState \in \{ECN_CE_RCVD, ECN_ECE_RCVD\}$ 时，判定存在 ECN 拥塞标记。与基于事件率统计的方法不同，Swift 直接检查协议栈的 ECN 状态字段，避免了维护时间戳队列的额外开销和事件率阈值的参数敏感性问题。需要强调的是，该分支成立的前提是瓶颈队列管理器确实执行 CE 标记：若瓶颈使用不标记 CE 的先进先出队列，则 $ecnState$ 不会进入 CE/ECE 状态，ECN 分支恒不触发。第4.1节据此将瓶颈默认配置为启用 ECN 标记的 RED 队列，并对全部对比协议对称开启 ECN，同时说明本文所引用的归档实验产物尚未包含该配置。

连续递减保护机制：为防止在突发拥塞期间窗口被过度收缩，Swift 维护一个连续递减计数器 d 。每次触发拥塞响应时 d 自增；仅当窗口真正发生增长时 d 才被重置为零——在降窗后的冻结期内窗口保持不变，因此 d 不被清零，从而使保护逻辑对连续到达的拥塞信号保持记忆。当 d 超过最大连续递减次数 $D_{max} = 3$ 时，拥塞响应被抑制——保持当前窗口不变，避免因连续乘性递减导致窗口收缩至过小值。连续 3 次丢包响应后窗口仅剩 $0.70^3 \approx 34.3\%$ ，已强于传统算法单次窗口减半的效果，进一步缩减的边际收益递减。

完整的多信号融合拥塞检测算法如算法1所示。

3.4 差异化拥塞响应机制

传统算法（如 NewReno、CUBIC）对所有拥塞信号采用统一的窗口减半策略（保留因子 $\rho = 0.5$ ），这种“一刀切”的方法无法区分轻度和严重拥塞，在轻度拥塞时响应过度（不必要地降低吞吐量），在严重拥塞时响应不足。Swift 根据拥塞类型设计差异化的窗口保留因子：

Algorithm 1 Multi-Signal Fusion Congestion Detection

Require: Observation obs , consecutive decrease counter d
Ensure: ($is_congested$, $type$)

```

1: // Level-1: window-reduction callback (GetSsThresh)
2: if  $obs.calledFunc = 0$  then
3:   if  $obs.caState = CA\_LOSS$  then
4:     return ( $True$ , TIMEOUT)
5:   else if  $obs.ecnState \in \{CE\_RCVD, ECE\_RCVD\}$  or  $obs.caState = CA\_CWR$  then
6:     return ( $True$ , ECN)
7:   else
8:     return ( $True$ , LOSS)
9:   end if
10: end if
11: // Level-2: ECN state inspection
12: if  $obs.ecnState \in \{CE\_RCVD, ECE\_RCVD\}$  then
13:   return ( $True$ , ECN)
14: end if
15: return ( $False$ , NONE)

```

$$new_cwnd = \rho \cdot cwnd,$$

$$\rho = \begin{cases} 0.70 & \text{丢包 (LOSS)} \\ 0.75 & \text{ECN 标记} \\ 0.50 & \text{超时 (TIMEOUT)} \end{cases} \quad (3)$$

设计依据如下。(1) **拥塞信号的时序特性：**ECN 标记是最早的信号（队列超低阈值时触发），丢包较晚（队列溢出），超时最晚（连续丢包或路径问题）。早期信号应采用相对温和但足以排空队列的响应，晚期信号采用更激进的响应。(2) **信号可靠性：**丢包几乎不会误报（ $LR \approx 100$ ），ECN 的可靠性取决于设备配置，因此对丢包响应更坚决。(3) **吞吐量-延迟权衡：**丢包保留因子 0.70 高于传统的 0.5，配合 ECN 的提前预警机制实现更平滑的响应；当前实现将 ECN 保留因子设为 0.75，使早期拥塞预警能够触发 25% 的窗口收缩，以降低持续排队驻留风险；超时保留因子 0.50 与传统窗口减半一致，反映超时信号所指示的严重拥塞或路径故障。

慢启动阈值更新为 $new_ssthresh = \max(\rho \cdot \min(cwnd, BDP), new_cwnd)$ ：以 $\min(cwnd, BDP)$ 为基准可避免将阈值抬升至 BDP 以上而变相奖励队列驻留，同时以 new_cwnd 为下界保证拥塞避免阶段仍可缓慢上探。窗口下限约束为 $4 \times MSS$ 。

此外，当前实现在拥塞响应路径上还包含三项与信号不确定性直接相关的安全约束。(1) **降窗后冻结：**每次进入拥塞响应即将冻结计数器置为 4 个 ACK 事件，且该赋值先于连续递减下限判定执行，因此即使响应被 D_{max} 抑制，队列仍获得排空窗口。(2) **超时重入慢启动：**当拥塞类型为超时，除施加 $\rho = 0.50$ 外还重置慢启动标志与慢启动阶段最小 RTT 采样，使控制律回到指数上探而非在错误的 BDP 估计附近继续微调。(3) **陈旧动作丢弃：**由于 GetSsThresh 回调接收的是常量协议栈状态，Python 侧返回的 $cwnd$ 需缓存至下一次 IncreaseWindow 才能生效；若在此期间协议栈进入 CA_LOSS （重传超时已重置窗口），环境侧将直接丢弃该缓存动作，避免陈旧决策覆盖协议栈的丢包恢复

过程。差异化响应算法如算法2所示。

Algorithm 2 Differentiated Congestion Response

Require: Current $cwnd$, congestion type $type$, consecutive decrease counter d , BDP estimate bdp

Ensure: New window new_cwnd , new threshold $new_ssthresh$

```

1:  $d \leftarrow d + 1$ ; consecutive increases  $\leftarrow 0$ 
2:  $freeze \leftarrow 4$  {Post-decrease freeze, armed before the floor test}
3: if  $d > D_{max}$  then
4:    $new\_cwnd \leftarrow \max(cwnd, 4 \times MSS)$  {Freeze window}

5:    $new\_ssthresh \leftarrow new\_cwnd$ 
6:   return ( $new\_cwnd$ ,  $new\_ssthresh$ )
7: end if
8: if  $type = \text{LOSS}$  then
9:    $\rho \leftarrow 0.70$ 
10: else if  $type = \text{ECN}$  then
11:    $\rho \leftarrow 0.75$ 
12: else if  $type = \text{TIMEOUT}$  then
13:    $\rho \leftarrow 0.50$ ; re-enter slow start
14: end if
15:  $new\_cwnd \leftarrow \max(\rho \times cwnd, 4 \times MSS)$ 
16:  $new\_ssthresh \leftarrow \max(\rho \times \min(cwnd, bdp), new\_cwnd)$ 

17: return ( $new\_cwnd$ ,  $new\_ssthresh$ )
```

3.5 强化学习驱动的自适应参数调优

Swift 根据多种网络状态因素动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ (初始值 1.10), 控制拥塞避免阶段窗口增长速度。当前实现的调优逻辑基于 RTT 膨胀、奖励信号 EMA 和连续增长趋势三个维度。与固定阈值不同, RTT 膨胀阈值会随最小 RTT 增大而放宽, 使长 RTT 路径不会因为正常的 BDP 填充过程过早触发收缩。

基于最小 RTT 感知阈值的调整: 设 $r = lastRtt/minRtt$ 。Swift 首先根据 $minRtt$ 计算松弛量 $slack = 0.3 + 0.15\sqrt{\max(0, minRtt_{ms} - 1)}$, 并构造 $low = 1 + slack$ 、 $mid = 1 + 2slack$ 、 $high = 1 + 4slack$ 三个动态阈值。当 $r < low$ 时, 说明队列驻留较低, α 按 0.02 或 0.03 小步增加; 当 $low \leq r < mid$ 时, α 仅增加 0.01 以维持温和探测; 当 $r > high$ 时, 说明排队膨胀明显, α 按 0.02 或 0.03 收缩并重置连续增长计数器。

基于奖励基线相对比较的调整: 为捕获较长期性能趋势, 同时维护奖励信号的快速指数移动平均 $\bar{r}_t = (1 - \eta) \cdot \bar{r}_{t-1} + \eta \cdot r_t$ ($\eta = 0.15$) 与慢速基线 $b_t = (1 - \eta_b) \cdot b_{t-1} + \eta_b \cdot r_t$ ($\eta_b = 0.02$)。由于逐 ACK 奖励几乎恒为正 (每个确认段贡献 +0.5 的吞吐收益), 固定绝对阈值不携带调节信息; 因此仅当 $\bar{r} > b + m$ 时 $\alpha += 0.01$, 当 $\bar{r} < b - 4m$ 时 $\alpha -= 0.01$, 其中动态裕量 $m = \max(0.25, 0.1|b|)$, 非对称下界使丢包/超时的惩罚尖峰仍能迅速收缩 α 。C++ 环境侧的奖励在 ACK 事件上定义为 $r = \min(0.5 \text{ segmentsAcked}, 5) - p_{rtt}$, 其中仅当 $lastRtt/RTT_{min} > 1.5$ 时 $p_{rtt} = \min(0.3(lastRtt/RTT_{min} - 1.5), 3)$, 否则为零; 在窗口收缩回调上则给出固定惩罚: ECN 为 -3、超时 (CA_LOSS) 为 -20、其他丢包为 -10。由此奖励同时携带传输进度与排队/丢包代价信息, 而其绝对量级

不影响 α 调节——只有相对基线的变化量被使用。

基于连续增长趋势的调整: 窗口连续增长超过 8 次时, 说明当前路径仍具备可探测空间, $\alpha += 0.01$ 。所有调整均限制在 $[0.85, 1.30]$ 内, 以避免无线、蜂窝和广域网场景中的持续队列堆积。完整的 α 自适应过程如算法3所示。

Algorithm 3 RTT- and Reward-Aware Adaptive Parameter Tuning

Require: Current α , observation obs , consecutive increase count g , reward EMA $\bar{r}$, reward baseline b

Ensure: Updated α

```

1:  $r \leftarrow obs.lastRtt / obs.minRtt$ 
2:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, obs.minRtt_{ms} - 1)}$ 
3:  $low \leftarrow 1 + slack$ ;  $mid \leftarrow 1 + 2slack$ ;  $high \leftarrow 1 + 4slack$ 
4: if  $r < low$  then
5:    $\alpha \leftarrow \alpha + (obs.minRtt_{ms} > 5?0.02 : 0.03)$ ;  $g \leftarrow g + 1$ 
6: else if  $r < mid$  then
7:    $\alpha \leftarrow \alpha + 0.01$ 
8: else if  $r > high$  then
9:    $\alpha \leftarrow \alpha - (obs.minRtt_{ms} > 5?0.03 : 0.02)$ ;  $g \leftarrow 0$ 
10: end if
11:  $m \leftarrow \max(0.25, 0.1|b|)$ 
12: if  $\bar{r} > b + m$  then
13:    $\alpha \leftarrow \alpha + 0.01$ 
14: else if  $\bar{r} < b - 4m$  then
15:    $\alpha \leftarrow \alpha - 0.01$ 
16: end if
17: if  $g > 8$  then
18:    $\alpha \leftarrow \alpha + 0.01$ 
19: end if
20:  $\alpha \leftarrow \text{clamp}(\alpha, 0.85, 1.30)$ 
```

3.6 融合控制策略

Swift 将 BDP 估计、目标窗口渐进逼近和安全边界约束相结合。BDP 估计采用窗口最大值滤波器 (Windowed Max-Filter): 投递速率由滑动时间窗口内的累计确认字节数计算, $DR = \Delta \text{bytes}_{acked} / \Delta t$, 时间窗口跨度取 $\min(\max(2 \times RTT_{min}, 5 \text{ ms}), 1 \text{ s})$, 使速率反映 ACK 时钟 (即瓶颈排空速率) 而非单个 ACK 的载荷 (后者会按每 RTT 的 ACK 数量成倍低估带宽); 速率样本存入容量为 $W_{bw} = 40$ 的滑动窗口队列, 取窗口内最大值 BW_{max} 作为瓶颈带宽估计; RTT 采样窗口容量为 $W_{rtt} = 40$, 并持续跟踪全局最小 RTT。BDP 计算为 $BDP = BW_{max} \times RTT_{min}$; 当 BDP 估计无效时回退至当前窗口值。BDP 估计算法如算法4所示。

慢启动阶段 ($cwnd < ssthresh$): 目标窗口 $target = 2 \times BDP$ (下限 $10 \times MSS$)。Swift 采用 HyStart 风格慢启动退出: 当当前 RTT 超过慢启动阶段最小 RTT 的动态阈值时, 提前退出慢启动以避免首轮 BDP 填充后的突发排队; 该阈值随最小 RTT 从 1.25 放宽到 1.40, 使广域网和卫星链路不因正常长 RTT 特性过早退出。标准增量为 $segmentsAcked \times MSS$; 仅当当前窗口明显低于 BDP 且 RTT 尚未膨胀时, 增量才提升为 $2 \times segmentsAcked \times MSS$ 。

拥塞避免阶段 ($cwnd \geq ssthresh$): 当前实现先计算目标速率窗口 $target = \alpha \times BDP$ (BDP 无效时回退到当前窗口), 再以有界步长向目标靠拢, 而不是直接把窗口跳变到目标值。当 $target > cwnd$ 时, 窗口按

Algorithm 4 Windowed Max-Filter BDP Estimation

Require: Flow state *state*, observation *obs***Ensure:** BDP estimate

```
1:  $state.bytes_{acked} \leftarrow state.bytes_{acked} + obs.segAked \times obs.segSize$ 
2:  $W \leftarrow \min(\max(2 \times RTT_{min}, 5 \text{ ms}), 1 \text{ s})$  {Rate window span}
3:  $dr \leftarrow \Delta bytes_{acked} / \Delta t$  over the last  $W$  {Delivery rate}
4:  $state.bw\_samples.append(dr)$  {Sliding window, capacity  $W_{bw} = 40$ }
5:  $BW_{max} \leftarrow \max(state.bw\_samples)$ 
6:  $RTT_{min} \leftarrow \min(state.min\_rtt, obs.minRtt)$ 
7:  $bdp \leftarrow BW_{max} \times RTT_{min}$ 
8: if  $bdp \leq 0$  then
9:    $bdp \leftarrow \max(obs.cwnd, 1)$  {Fallback}
10: end if
11: return  $bdp$ 
```

$\max(\gamma \times MSS, MSS, (target - cwnd)/16)$ 上探, 并以 $target + \gamma \times MSS$ 为本次上限, 从而在约 16 个 ACK 事件内完成 BDP 填充而不产生超调; 当 $target < cwnd$ 时, 每个 ACK 事件按 $\max((cwnd - target)/2, MSS)$ 回落且不低于 $target$, 以主动排空队列; 当二者相等时, 按 $\gamma \times MSS$ 进行 Reno 式加性增长。当前实现取 $\gamma = 1.0$ 。此外, 若降窗冻结计数器非零, 本次事件直接返回当前窗口并将计数器减一, 冻结期内不做任何增长。

管道利用率感知加速与安全边界: 管道利用率 $u = bytesInFlight/cwnd$ 用于识别未充分填充的链路。该增量仅在 RTT 未明显膨胀 ($lastRtt < 1.3 RTT_{min}$) 时生效: 当 $u < 0.4$ 时, 短 RTT 路径追加 $2 \times MSS$, 最小 RTT 超过 5 ms 的长 RTT 路径仅追加 $1 \times MSS$; 当 $0.4 \leq u < 0.5$ 时, 只有短 RTT 路径再追加 $1 \times MSS$ 。这一分级设计避免了在无线、蜂窝和广域网场景中因盲目加速而累积队列。最终窗口被约束在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内, 避免 BDP 估计陈旧时窗口失控。融合控制策略的完整算法如算法5所示。

3.7 每流独立状态管理

Swift 为每个 TCP 连接维护独立状态, 通过 socketUid 为键的哈希表实现 $O(1)$ 查找。每流状态包括四个子模块: (1) **带宽与 RTT 估计模块**——容量 $W_{bw} = 40$ 的投递速率滑动窗口队列和容量 $W_{rtt} = 40$ 的 RTT 采样队列, 支持窗口最大值带宽估计、全局最小 RTT 追踪和慢启动阶段最小 RTT 采样; (2) **拥塞控制计数器**——连续递减计数器 d 、连续增长计数器 g 、累计丢包和 ECN 事件计数, 以及降窗后的 ACK 冻结计数; (3) **性能指标聚合**——吞吐量指数移动平均、当前 BDP 估计、上一步窗口和时间戳; (4) **自适应参数**—— α (初始 1.10)、 γ (默认 1.0) 和慢启动标志位。

状态管理遵循延迟初始化 (首次访问时创建) 和增量更新 (仅更新变化字段) 两个原则。每流独立管理避免了多流并发场景下的流间干扰, 支持差异化控制。奖励 EMA (平滑因子 $\eta = 0.15$) 与慢速基线 ($\eta_b = 0.02$) 由每个智能体实例独立维护——每个 TCP 连接对应一个智能体实例, 避免流间奖励串扰。

Algorithm 5 Hybrid Rate-Window Fusion Control

Require: Observation *obs*, parameter α , BDP estimate bdp , freeze counter f **Ensure:** New window new_cwnd

```
1: if  $f > 0$  then
2:    $f \leftarrow f - 1$ ;
3:   return  $obs.cwnd$  {Post-decrease freeze}
4: end if
5:  $d \leftarrow 0$  {Reset the consecutive-decrease counter only on growth}
6: if  $obs.cwnd < obs.ssthresh$  and flow is in slow start then
7:   Apply HyStart-style RTT-inflation exit (1.25/1.30/1.40 by  $RTT_{min}$ )
8:    $target \leftarrow \max(2 \times bdp, 10 \times MSS)$ 
9:    $\Delta \leftarrow obs.segmentsAked \times MSS$ 
10:  if  $obs.cwnd < 0.3 \times bdp$  and  $obs.lastRtt < 1.2 RTT_{min}$  then
11:     $\Delta \leftarrow 2 \times obs.segmentsAked \times MSS$ 
12:  end if
13:   $new\_cwnd \leftarrow \min(obs.cwnd + \Delta, target)$ 
14: else
15:   $target \leftarrow \alpha \times bdp$ 
16:  if  $target > obs.cwnd$  then
17:     $step \leftarrow \max(\gamma \times MSS, MSS, (target - obs.cwnd)/16)$ 
18:     $new\_cwnd \leftarrow \min(obs.cwnd + step, target + \gamma \times MSS)$ 
19:  else if  $target < obs.cwnd$  then
20:     $new\_cwnd \leftarrow \min(\max(obs.cwnd - \max(\frac{obs.cwnd - target}{2}, MSS), target)$ 
21:  else
22:     $new\_cwnd \leftarrow obs.cwnd + \gamma \times MSS$ 
23:  end if
24: end if
25: if  $obs.lastRtt < 1.3 RTT_{min}$  then
26:   $u \leftarrow obs.bytesInFlight/obs.cwnd$ 
27:  if  $u < 0.4$  then
28:     $new\_cwnd \leftarrow new\_cwnd + (RTT_{min} > 5 \text{ ms} ? MSS : 2 \times MSS)$ 
29:  else if  $u < 0.5$  and  $RTT_{min} \leq 5 \text{ ms}$  then
30:     $new\_cwnd \leftarrow new\_cwnd + MSS$ 
31:  end if
32: end if
33:  $new\_cwnd \leftarrow \text{clamp}(new\_cwnd, 4 \times MSS, \max(4 \times bdp, 200 \times MSS))$ 
34: return  $new\_cwnd$ 
```

4 实验评估

4.1 仿真配置与实验矩阵

软硬件环境：ns-3.40 [21] 与 ns3-gym [22], Python 3.8, Intel i7-10700K (8 核) 处理器、32 GB 内存、Ubuntu 20.04。

拓扑与负载：使用 PointToPointDumbbellHelper 构造哑铃拓扑，左右各 3 个叶子节点 ($nLeaf = 3$) 经接入链路汇聚到一条共享瓶颈链路；每对叶子节点上运行一条 BulkSendHelper 长流，3 条流分别在 0、100 与 200 ms 错峰启动，仿真时长 20 s。MTU 取 1500 B，扣除 IP/TCP 头部与预留后应用数据单元 (MSS) 为 1440 B，初始窗口 10 个 MSS (RFC 6928)，启用 SACK。瓶颈队列长度按 $\max(BDP \cdot nLeaf / MTU, 100)$ 个报文配置，其中 BDP 由瓶颈速率与基础 RTT 算出，从而使缓冲深度随场景带宽和时延同步伸缩。

队列管理与 ECN：v3.0.0 参考配置将瓶颈队列设为 RED，标记下限 $MinTh = 0.3Q$ 、上限 $MaxTh = 0.9Q$ 、UseEcn=true，并对四种协议一律设置 TcpSocketBase::UseEcn=On，使 ECN 信号在对照组之间对称可得。需要明确指出的是，本节引用的归档实验产物由该修正之前的版本生成：当时瓶颈使用不执行 CE 标记的 PfifoFast 队列，且 ECN 仅对 TcpSwift 开启。因此在这些产物中 Swift 的 ECN 分支实际从未触发，下文报告的增益只能归因于 $\alpha \times BDP$ 目标窗口的有界逼近、差异化丢包/超时响应以及连续递减与冻结保护；ECN 路径的实际贡献须待全矩阵重跑后单独评估（见第5.3节）。

A/B 对照设置：两组实验共享完全相同的场景参数、拓扑与随机种子，仅 enable_udp_burst 取值不同：

- **纯 TCP 设置：**瓶颈上仅承载 3 条 TCP 长流；
- **UDP 突发设置：**在同一瓶颈上叠加一路 800 Mbps、50% 占空比 (On 0.5 s / Off 0.5 s)、报文长 1024 B 的 OnOff UDP 源，用以模拟视频、RPC、后台同步与备份等非响应式跨流量。

对比算法与评测指标：NewReno [2]、CUBIC [3] 与 BBR [6]。指标包括 3 条前向流的聚合有效吞吐量 (Goodput, Mbps)、逐包平均单向端到端延迟 (ms)、丢包率 (%) 以及流间 Jain 公平性指数 [23]。实验运行器支持 --num-seeds 多随机种子重复，产物按 `< 场景 >_< 协议 >_< 种子 >` 命名。

4.2 测量方法修正与数据筛选

反向流污染修正：既有汇总表 logs/plots/summary.csv 与 logs/plots-udp/summary.csv 在聚合时把反向 ACK 流 (10.2.x→10.1.x) 与前向数据流一并计入，由此产生两类系统性偏差：其一，延迟被计算为数据流与 ACK 流的均值，约等于 $(D_{data} + D_{prop})/2$ ，使数据面延迟被折半；其二，吞吐量被 ACK 流额外抬高约 1.7%。为消除该缺陷，本文从原始 FlowMonitor 产物重新导出仅含前向数据流的 KPI，结果提交于 logs/summary/kpi_forward.csv (288 条记录，

字段包含设置、场景、协议、瓶颈速率、基础单向时延、流数、汇聚端口、吞吐量、利用率、延迟、抖动、丢包率、Jain 指数与产物路径)。本节所有数值以该文件为唯一数据来源，其中吞吐量为 3 条前向长流之和，延迟为这些流的逐包平均单向时延，Jain 指数在 3 条流之间计算。

排除规则：对完整归档矩阵 (36 个场景 × 4 个协议 × 2 种设置) 执行数据质量审计，得到四条排除规则，全部记录于 logs/error.txt 的“AUDIT PASS 2026-08-20”条目中：

- **规则 A (指标定义缺陷)：**两份绘图汇总表的吞吐量与延迟列由 kpi_forward.csv 取代。
- **规则 B (仿真器版本混用)：**27 个 (设置, 场景) 组合内部不同协议的汇聚端口分别为 5000 与 50000，表明产物来自两个 sim.cc 版本，跨协议对比不成立，相关场景整体排除。
- **规则 C (重复与错标配置)：**dc_oversub_10to1 与 congested_heavy、satellite_leo 与 lte_good 的配置完全相同且产物逐字节一致，保留后者。其中 satellite_leo 实际被配置为 100/50 Mbps、5/20 ms 链路，并不具备 LEO 卫星链路特征，属标签错误；因此本文不基于该数据集给出任何 LEO 卫星结论，GEO 卫星场景 (satellite_geo) 不受影响。
- **规则 D (基线退化)：**在微秒级 RTT 高速率路径上 BBR 吞吐量不足瓶颈容量的 1% (约 2 Mbps)，其成因是 ns-3.40 中 BBR 的 pacing 与 RTT 分辨率失配，而非拥塞控制策略的结果；据此在 intra_rack_10g 与 intra_rack_25g 中排除 BBR，这两个场景退化为三协议对比。

经上述规则筛选后，纯 TCP 设置保留 19 个场景，UDP 突发设置保留 15 个场景 (后者为前者的真子集)，合计 132 条可用于跨协议对比的记录。场景配置如表2所示。为便于组织讨论，按路径特征将 19 个场景归为四组：**G1** 为微秒级 RTT 高速链路，**G2** 为千兆级近端与城域/跨域链路，**G3** 为无线与蜂窝接入链路，**G4** 为 GEO 卫星链路。

4.3 纯 TCP 设置下的稳态性能

表3、表4与表5分别给出 19 个洁净场景的聚合吞吐量、平均单向延迟与丢包率。

吞吐量：在饱和链路上 Swift 与两种基于丢包的基线完全等价。**G1** 的 3 个场景以及 asymmetric_high、congested_heavy、symmetric_low、dc_100m、dc_500m 中，Swift 与 NewReno/CUBIC 的吞吐量在三位小数上一致 (相对差 +0.000%)。**G3** 的 8 个无线与蜂窝场景差异均在 ±0.5% 以内 (最大偏离为 lte_good 的 +0.45% 与 wifi_ax 的 -0.31%)。GEO 卫星场景 (**G4**) 是唯一出现显著差距的正向结果：Swift 为 8.57 Mbps，相对 NewReno (5.61)、CUBIC (7.59) 与 BBR (2.81) 分别提升 52.8%、12.9% 与 205.0%，相对 NewReno/CUBIC 均值提升 29.8%；其机理是零丢包避免了在 ≈640 ms 往返时延上以整个 RTT 为代价的重传恢复。反向地，cross_dc_wan 是本文最主要的负向结果：Swift 仅 709.72 Mbps，较 NewReno

表 2: 审计后保留的 19 个纯 TCP 场景配置

组	场景	瓶颈 (Mbps)	基础单向时延 (ms)
G1	intra_rack_10g [‡]	10000	0.004
G1	intra_rack_25g [‡]	25000	0.004
G1	leaf_spine_20g	20000	0.009
G2	asymmetric_high	1000	0.012
G2	congested_heavy	1000	0.009
G2	symmetric_low	1000	0.030
G2	dc_100m	100	0.009
G2	dc_500m	500	0.009
G2	wan_metro	1000	2.20
G2	cross_dc_wan [†]	1000	5.02
G3	wifi_ac [†]	400	7.00
G3	wifi_ax [†]	600	5.00
G3	wifi_n	50	14.0
G3	wifi_legacy	10	30.0
G3	lte_good	50	30.0
G3	lte_poor	10	70.0
G3	nr_5g_edge	100	14.0
G3	nr_5g_embb [†]	500	7.00
G4	satellite_geo	10	320.0

[†] 该场景不在 UDP 突发设置的 15 个洁净场景之列；[‡] 该场景按规则 D 排除 BBR 基线。

表 3: 纯 TCP 设置：聚合有效吞吐量 (Mbps)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	10038.0	10038.0	10038.0	—
intra_rack_25g	25095.0	25095.0	25095.0	—
leaf_spine_20g	20059.2	20059.2	20059.1	7414.7
asymmetric_high	1003.6	1003.6	1003.6	510.16
congested_heavy	1003.6	1003.6	1003.6	507.86
symmetric_low	1003.6	1003.6	1003.6	482.83
dc_100m	100.36	100.36	100.36	51.53
dc_500m	501.81	501.81	501.81	257.07
wan_metro	1000.3	1001.7	1002.3	1005.0
cross_dc_wan	709.72	998.38	1000.0	1001.5
wifi_ac	398.66	398.91	399.69	398.91
wifi_ax	598.37	599.38	600.26	600.92
wifi_n	49.88	49.80	49.88	48.92
wifi_legacy	9.95	9.95	9.95	9.85
lte_good	49.46	49.24	49.39	46.33
lte_poor	9.81	9.77	9.81	9.28
nr_5g_edge	99.63	99.35	99.63	98.84
nr_5g_embb	497.66	498.50	499.52	499.93
satellite_geo	8.57	5.61	7.59	2.81

表 4: 纯 TCP 设置：前向流平均单向延迟 (ms)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.71	3.94	4.51	—
intra_rack_25g	0.28	3.79	4.39	—
leaf_spine_20g	0.35	3.81	4.44	0.01
asymmetric_high	6.97	5.87	5.49	0.13
congested_heavy	6.97	5.87	5.49	0.13
symmetric_low	6.95	5.94	5.52	0.07
dc_100m	35.42	16.25	15.48	1.37
dc_500m	10.72	6.98	6.67	0.16
wan_metro	4.93	7.02	7.18	5.26
cross_dc_wan	5.04	8.88	9.56	10.73
wifi_ac	10.86	12.10	13.02	16.12
wifi_ax	6.93	9.56	10.17	12.28
wifi_n	78.16	38.22	39.76	73.95
wifi_legacy	242.23	135.63	164.09	111.73
lte_good	80.61	50.51	70.10	66.80
lte_poor	310.79	174.19	188.00	172.22
nr_5g_edge	44.69	26.63	29.22	64.42
nr_5g_embb	7.33	11.48	12.43	17.38
satellite_geo	391.42	365.15	420.22	612.08

表 5: 纯 TCP 设置：丢包率 (%)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.00	0.01	0.00	—
intra_rack_25g	0.00	0.00	0.01	—
leaf_spine_20g	0.00	0.00	0.01	0.00
asymmetric_high	0.00	0.00	0.00	0.00
congested_heavy	0.00	0.00	0.00	0.00
symmetric_low	0.00	0.00	0.00	0.00
dc_100m	0.00	0.07	0.02	0.00
dc_500m	0.00	0.01	0.00	0.00
wan_metro	0.00	0.00	0.00	0.00
cross_dc_wan	0.00	0.00	0.00	0.00
wifi_ac	0.00	0.00	0.00	0.03
wifi_ax	0.00	0.00	0.00	0.01
wifi_n	0.00	0.04	0.05	0.14
wifi_legacy	0.00	0.13	2.11	0.07
lte_good	0.00	0.03	1.05	0.77
lte_poor	0.00	0.17	1.80	1.79
nr_5g_edge	0.00	0.01	0.02	0.08
nr_5g_embb	0.00	0.00	0.00	0.02
satellite_geo	0.00	0.59	1.26	9.83

数值按两位小数四舍五入。严格为零的场景数：Swift 19/19；NewReno 与 CUBIC 均为 0；BBR 为 6，但其 6 个零丢包场景的链路利用率仅 0.37~0.52。

低 28.9%、较 CUBIC 低 29.0%（详见第 5.3 节）。BBR 在千兆级链路上系统性欠利用（利用率 0.48~0.52），在 leaf_spine_20g 上仅取得 0.37 的利用率；但在 cross_dc_wan、wan_metro、nr_5g_embb 与 wifi_ax 上 BBR 的吞吐量略高于 Swift（差距 0.4%~41% 不等），本文不回避这一点。

延迟：Swift 的延迟表现具有清晰的场景依赖性，且与瓶颈缓冲深度相对 BDP 的比例直接相关。在 G1 的微秒级 RTT 路径上，排队时延几乎构成端到端时延的全部，Swift 把平均单向延迟从 3.79~4.51 ms 降至 0.28~0.71 ms，相对 NewReno/CUBIC 分别降低 82.1%/84.3%（10G）、92.6%/93.6%（25G）与 90.9%/92.2%（20G 叶脊），且吞吐量未有任何损失。在中速接入与城域链路上 Swift 同样占优：nr_5g_embb 降低 36.2%/41.0%、wifi_ax 降低 27.5%/31.9%、wan_metro 降低 29.8%/31.3%、wifi_ac 降低 10.2%/16.6%，cross_dc_wan 亦取得全场最低延迟（5.04 ms 对 8.88/9.56/10.73 ms）。但在低速率或长 RTT 且缓冲深度远大于 BDP 的接入链路上，Swift 以队列驻留换取零丢包，延迟相对更优的丢包类基线出现抬升：dc_100m 为 +128.8%、wifi_n +104.5%、wifi_legacy +78.6%、lte_poor +78.4%、nr_5g_edge +67.8%、dc_500m +60.6%、lte_good +59.6%，千兆级饱和链路约 +26%，GEO 卫星为 +7.2%。该现象的机理与量化边界在第 5.3 节讨论。

丢包与公平性：Swift 在全部 19 个场景中丢包率严格为 0.00%，是四种协议中唯一做到这一点的算法；NewReno 与 CUBIC 没有任何一个场景达到严格零丢包，BBR 虽在 6 个场景中严格为零，但这些场景的链路利用率仅 0.37~0.52，即以牺牲近半瓶颈容量换取无丢包。基线的最大丢包率分别为 NewReno 0.59%（satellite_geo）、CUBIC 2.11%（wifi_legacy）与 BBR 9.83%（satellite_geo）。公平性方面，Swift 的 Jain 指数在所有场景均为 1.00，NewReno 均值 0.988（最低 0.82）、CUBIC 均值 0.993（最低 0.91）、BBR 均值 0.812（最低 0.38，见 wifi_n）。聚合统计见表 6。

表 6: 丢包率与公平性聚合统计 (洁净场景集)

设置	协议	零丢包	最大	均值	Jain 均	Jain 最小
纯 TCP $n = 19$	Swift	19/19	0.000	0.000	1.000	1.00
	NewReno	0/19	0.591	0.058	0.988	0.82
	CUBIC	0/19	2.115	0.334	0.993	0.91
	BBR	6/17	9.828	0.750	0.812	0.38
UDP 突发 $n = 15$	Swift	15/15	0.000	0.000	1.000	1.00
	NewReno	0/15	0.594	0.107	0.998	0.98
	CUBIC	0/15	7.714	1.017	0.997	0.97
	BBR	6/13	12.338	1.742	0.844	0.48

”零丢包”统计丢包率严格为 0 的场景数；BBR 的分母为 17/13，因规则 D 在两个微秒级 RTT 场景中排除该基线。丢包率单位为%。

4.4 深入分析

4.4.1 分组讨论

G1——微秒级 RTT 高速链路：瓶颈 ≥ 10 Gbps，基础单向时延仅 $4\sim 9\ \mu\text{s}$ ，因此端到端时延几乎全部来自瓶颈排队。NewReno 与 CUBIC 按”增长至丢包”的范式运行，稳态下把队列推至接近满载，故延迟停留在 $3.8\sim 4.5\ \text{ms}$ ；Swift 的目标窗口被 $\alpha \times BDP$ ($\alpha \leq 1.30$) 显式约束，并在超过目标时按超额一半的速率回落，因此在维持完全相同吞吐量的同时把队列压到极低水位。该组是 Swift”等吞吐、低延迟”性质最清晰的证据。

G2——千兆级近端与城域/跨域链路：在 3 个千兆级饱和场景 (asymmetric_high、congested_heavy、symmetric_low) 中三种算法吞吐量一致，Swift 延迟高出约 26%；在带宽更低的 dc_500m 与 dc_100m 上该抬升分别达 60.6% 与 128.8%。原因是瓶颈缓冲深度按 $BDP \cdot n_{\text{Leaf}}/MTU$ 配置但下限为 100 个报文，当 BDP 很小时该下限主导， $\alpha > 1$ 的目标窗口足以在其中形成稳定驻留。随着传播时延增大 (wan_metro 2.2 ms、cross_dc_wan 5.02 ms)，BDP 上升、缓冲相对变浅，Swift 转为延迟最优 (分别降低 29.8% 与 43.2%)；但在 cross_dc_wan 上这一延迟优势以近 29% 的吞吐损失为代价，属于典型的带宽估计不足而非有意的延迟折中。

G3——无线与蜂窝接入链路：8 个场景的吞吐量差异均在 $\pm 0.5\%$ 以内，区分度体现在丢包率与延迟上。CUBIC 在 wifi_legacy (2.11%)、lte_poor (1.80%)、lte_good (1.05%) 上出现显著丢包，BBR 在 lte_poor 上达 1.79% 且 Jain 指数低至 $0.38\sim 0.59$ ；Swift 保持严格零丢包与理想公平性。延迟方面该组呈现两极：带宽较高的 nr_5g_embb、wifi_ax、wifi_ac 上 Swift 延迟最低，而带宽较低的 wifi_n、wifi_legacy、lte_poor、nr_5g_edge 上 Swift 延迟最高，与 G2 中观察到的”浅 BDP+ 深缓冲”机理一致。

G4——GEO 卫星链路：单向时延 320 ms、瓶颈 10 Mbps，重传代价极高。Swift 以 8.57 Mbps 取得该组最高吞吐量，并同时保持零丢包与 1.00 的 Jain 指数；NewReno 受 0.59% 丢包与 0.82 的公平性拖累仅 5.61 Mbps，BBR 因 9.83% 丢包退化至 2.81 Mbps。该场景中 Swift 的延迟仅比 NewReno 高 7.2%，却低于 CUBIC 6.9% 与 BBR 36.1%，表明在长 RTT 路径上其吞吐优势并非以延迟为代价换取。

4.4.2 机制一致性与证据边界

差异化响应：当前实现将拥塞信号分为丢包、ECN 与超时三类并分别采用 0.70、0.75 与 0.50 的保留因子。在归档产物中瓶颈未执行 CE 标记，故 ECN 分支未被触发，观测到的零丢包只能由 $\alpha \times BDP$ 目标窗口的主动回落、 $\rho_{\text{loss}} = 0.70$ 的温和收缩以及超时时的 $\rho = 0.50$ 加慢启动重入共同解释。ECN 分支在 v3.0.0 配置下才具备可观测性，因此本文不把 ECN 响应写作已验证的定量贡献。

连续递减保护与冻结： $D_{\text{max}} = 3$ 与降窗后 4 个 ACK 事件冻结共同限制了短时间内的重复收缩与随后的快速反弹。由于仓库未保存独立的消融重跑结果，本文只把该机制作为定性解释，不给出无来源的百分比收益。

α 自适应： $\alpha \in [0.85, 1.30]$ 、初值 1.10，由最小 RTT 感知阈值、奖励基线相对偏移与持续增长趋势共同调节。该机制的可解释后果在数据中有一致体现： α 上界 1.30 意味着稳态目标窗口最多超出 BDP 约 30%，这既解释了浅 BDP 链路上的队列驻留，也解释了在长 RTT 链路上窗口不会无界膨胀。由于日志中没有固定 α 、固定 RTT 阈值或不同 α 边界的完整对照结果，本文不给出参数扫描数字。

4.5 UDP 突发跨流量鲁棒性评估

异构终端与共享链路普遍承载突发性非响应式 UDP 流量 (视频、RPC、后台同步、备份、游戏控制等)。本节在严格 A/B 对照下比较四种协议在 800 Mbps、50% 占空比 UDP 突发干扰下的表现，数据同样取自 logs/summary/kpi_forward.csv 的 udp_burst 设置，洁净场景为 15 个。

4.5.1 聚合鲁棒性指标

以同一场景的纯 TCP 结果为基准，定义吞吐量保持率为 $T_{\text{udp}}/T_{\text{tcp}}$ 、延迟膨胀为 $D_{\text{udp}}/D_{\text{tcp}} - 1$ ，在 15 个成对场景上聚合得到表 7。

表 7: UDP 突发跨流量鲁棒性聚合指标 (成对场景, $n = 15$)

协议	保持率均值	保持率最小	延迟膨胀	零丢包
Swift	80.54%	66.51%	+14.33%	15/15
NewReno	75.86%	35.47%	+4.32%	0/15
CUBIC	78.78%	53.75%	+3.22%	0/15
BBR	39.07%	9.69%	+50.78%	6/13

BBR 的分母为 13，因规则 D 在两个微秒级 RTT 场景中排除该基线；三项保持率最小值均出现在 satellite_geo (BBR 除外，其最小值出现在 wifi_n)。

三项观察如下。第一，Swift 的吞吐量保持率最高 (80.54%)，且最坏场景仍保持 66.51%，而 NewReno 与 BBR 的最坏场景分别跌至 35.47% 与 9.69%。第二，Swift 是唯一在全部 15 个场景中保持严格零丢包的算法；CUBIC 在 lte_poor 上从 1.80% 恶化至 7.71%，在 wifi_legacy 上从 2.11% 恶化至 4.00%，BBR 在 satellite_geo 上从 9.83% 恶化至 12.34%。第三，Swift 的延迟膨胀 (+14.33%) 高于 NewReno (+4.32%) 与 CUBIC (+3.22%)，这与稳态结论一致： $\alpha \times BDP$ 目标窗口在跨流量抬高 RTT 噪声时倾向于保留更多在途字节；但 BBR 的 +50.78% 说明这一代价在四者中并非最高。

4.5.2 场景级对照

表8给出 15 个洁净场景的吞吐量与延迟。Swift 在其中 14 个场景取得最高吞吐量，唯一例外是 wan_metro (801.06 Mbps 对 CUBIC 802.95 与 NewReno 801.87，相对最优基线低 0.24%)。优势最显著的是 GEO 卫星场景：Swift 5.70 Mbps 对 NewReno 1.99、CUBIC 4.08 与 BBR 1.80，分别提升 186.4%、39.7% 与 216.7%，相对 NewReno/CUBIC 均值提升 87.8%——即纯 TCP 设置下的 29.8% 优势在跨流量干扰下扩大约三倍。中低速蜂窝与 WiFi 场景亦呈现一致趋势：lte_good 提升 12.7% (对 NewReno) 与 2.7% (对 CUBIC)、nr_5g_edge 提升 9.1% 与 2.1%、wifi_n 提升 8.0% 与 2.1%、lte_poor 提升 15.7% 与 10.5%。

BBR 在跨流量下出现系统性崩塌：在千兆级链路上吞吐量从 482.83~510.16 Mbps 跌至 49.08~56.29 Mbps (降幅约 89%)，wifi_n 从 48.92 Mbps 跌至 4.74 Mbps (降幅 90.3%)，dc_500m 从 257.07 跌至 27.95 Mbps (降幅 89.1%)。这与已有关于 BBR 带宽探测在竞争流量下失真的实测报告一致 [8]。

4.5.3 低速与无线接入链路的零丢包验证

跨流量干扰把基线的丢包问题显著放大，而 Swift 在这些条件下仍保持严格零丢包，构成其鲁棒性的最直接证据：lte_poor 中 CUBIC 丢包率升至 7.71%、BBR 升至 4.14%，Swift 为 0.00% 且吞吐量最高 (7.35 Mbps)；wifi_legacy 中 CUBIC 升至 4.00%、BBR 升至 2.64%，Swift 为 0.00% (7.42 Mbps)；lte_good 中 CUBIC 升至 2.96%、BBR 为 0.93%，Swift 为 0.00% (37.21 Mbps，相对 BBR 的 16.69 Mbps 提升 1.23 倍)；wifi_n 中 BBR 升至 2.27% 且吞吐量崩塌至 4.74 Mbps，Swift 为 0.00% (37.88 Mbps)。这四个场景的瓶颈带宽介于 10~50 Mbps，基础单向时延 14~70 ms，说明该性质在低速率、长时延且丢包背景本身较高的接入链路上同样成立。

4.5.4 鲁棒性机理分析

在归档配置下，Swift 的跨流量鲁棒性可归因于两项机制。第一，**BDP 窗口最大值滤波**对 UDP 突发引入的瞬时速率波动具有低通特性：容量 40 的速率样本窗口取最大值，使带宽估计锚定在突发间歇期的真实瓶颈能力附近，而不像 BBR 的周期性探测那样被竞争流量直接污染，这解释了两者的保持率 80.54% 与 39.07% 的巨大差距。第二，**目标窗口的有界逼近与差异化收缩**使窗口在突发到来时按超额一半的速率回落、在突发退去时于约 16 个 ACK 事件内回填，既避免了乘性减半造成的吞吐塌陷，也避免了持续过冲导致的溢出丢包。需要重申的是，ECN 温和响应 ($\rho_{ecn} = 0.75$) 在这些归档产物中并未参与，其对鲁棒性的贡献须由后续在 RED/ECN 配置下的重跑给出。

5 讨论

5.1 理论分析

收敛性：Swift 的窗口动态可用 Lyapunov 函数 $V = \frac{1}{2}(cwnd - cwnd^*)^2$ 进行定性分析，其中 $cwnd^* \approx BDP$ 。在拥塞避免阶段，窗口不再直接跳变到 $\alpha \times BDP$ ，而是以有界步长向目标窗口渐进逼近；当检测到 ECN、丢

包或超时，窗口分别按 0.75、0.70 和 0.50 的保留因子收缩。连续递减保护 ($D_{max} = 3$) 与降窗后 4 个 ACK 事件的冻结机制进一步抑制重复收缩与快速反弹。 α 的边界约束 [0.85, 1.30] 和小步长调整保证参数不会在单个 RTT 内发生突变。

复杂度与计算开销：当前实现采用基于规则的确定性策略，单次决策仅包含固定数量的状态读取、分支判断、窗口最大值更新和哈希表查询，时间复杂度为 $O(1)$ ；每流状态主要由带宽样本队列、RTT 样本队列、拥塞计数器、窗口历史和少量自适应参数构成，空间复杂度为 $O(n \cdot (W_{bw} + W_{rtt}))$ ，其中 n 为并发流数， $W_{bw} = 40$ 、 $W_{rtt} = 40$ 。由于当前仓库未保存独立的推理延迟、CPU 利用率和内存压测日志，本文仅给出复杂度级别分析，不声称具体硬件平台上的毫秒级延迟或大规模并发资源占用数字。

信息论视角：多信号融合的两级优先级等价于按似然比排序。设网络拥塞状态 $C \in \{0, 1\}$ ，各信号条件独立，则联合似然比 $LR(\mathbf{S}) = \prod_i P(S_i | C = 1) / P(S_i | C = 0)$ 。丢包/超时 $LR \approx 100$ ，ECN $LR \approx 20$ 。优先使用高似然比信号可最小化贝叶斯误检概率 P_e 。连续递减保护机制从信息论角度可理解为对连续同类信号的置信度递减——第 k 次连续信号的增量信息量 $I_k \propto 1/k$ ，当 $k > D_{max}$ 时信息增益不足以证明进一步缩减的合理性。

最优控制视角：差异化保留因子可从最优控制理论分析。严重拥塞（超时，连续丢包或路径故障）需最大窗口缩减 ($\rho = 0.50$) 快速清空队列；中度拥塞（丢包，队列溢出）采用中等缩减 ($\rho = 0.70$)；轻度拥塞（ECN，队列开始积累）采用相对温和但足以抑制队列扩张的缩减 ($\rho = 0.75$)。Swift 的三类差异化设计近似了不同拥塞程度下的最优控制输入，响应强度与拥塞严重程度正相关。

5.2 与相关工作的详细对比

vs. Gemini [19]：两者都把“控制逻辑白盒化、让学习只作用于参数”作为出发点，但适配回路的位置和时间尺度不同。Gemini 的 Fusion 固定在内核数据面，Booster 在带外以分钟级、流组粒度用贝叶斯优化搜索参数，因此单条流在其生命周期内使用的是一组静态参数；Swift 把整个决策过程置于协议栈之外，并在每个窗口调整回调上求解， α 与目标窗口对单条流的即时 RTT 与奖励变化作出反应。这一差异带来两个可比较的后果：Swift 可以读取 ECN 与 CA 状态机从而区分 ECE/CWR 触发的收缩与真实丢包，是 Gemini 的流级统计量无法提供的信息；但 Swift 为此付出每事件一次跨进程往返的代价，其当前形态适用于研究与仿真平台，尚不具备 Gemini 已验证的生产部署性质。

vs. Aurora [17]：Swift 采用 11 维有效状态子空间 (Aurora 常用 10 维观测)，并引入 ECN 和 CA 状态参数。Swift 采用基于规则的确定性决策而非深度神经网络，无需训练即可运行，决策路径可逐分支解释。Aurora 的优势在于神经网络的表示能力，代价是训练数据收集、模型更新与推理开销；Swift 则以可解释规则和协议栈内部信号换取工程可控性。本文不主张 Swift 在策略表示能力上优于 Aurora。

表 8: UDP 突发设置下 15 个洁净场景的吞吐量与延迟（前向流聚合）

场景	吞吐量 (Mbps)				平均单向延迟 (ms)			
	Swift	NewReno	CUBIC	BBR	Swift	NewReno	CUBIC	BBR
intra_rack_10g	9643.3	9643.3	9643.3	—	0.74	3.90	4.45	—
intra_rack_25g	24700.6	24700.6	24700.5	—	0.29	3.77	4.38	—
leaf_spine_20g	19664.4	19664.4	19664.3	7426.0	0.35	3.77	4.30	0.01
asymmetric_high	806.06	804.13	804.53	55.88	8.28	5.93	5.20	0.17
congested_heavy	806.06	804.21	804.53	56.29	8.28	5.93	5.20	0.17
symmetric_low	794.62	792.80	792.96	49.08	8.43	6.24	5.34	0.10
dc_100m	76.52	75.51	76.52	12.80	41.33	15.75	15.04	6.00
dc_500m	382.56	382.56	382.56	27.95	11.16	7.21	6.52	0.28
wan_metro	801.06	801.87	802.95	723.51	6.71	6.94	6.72	5.38
wifi_n	37.88	35.08	37.09	4.74	94.62	41.65	42.01	106.48
wifi_legacy	7.42	6.64	7.32	4.68	241.55	144.12	173.36	151.03
lte_good	37.21	33.03	36.23	16.69	97.94	59.47	88.21	114.43
lte_poor	7.35	6.35	6.65	6.20	304.53	185.39	262.62	210.83
nr_5g_edge	75.71	69.40	74.16	43.21	51.77	27.87	28.52	49.51
satellite_geo	5.70	1.99	4.08	1.80	530.71	427.11	437.54	541.41

表 9: Swift 与主要相关算法的特性对比

特性	Swift	Gemini	Aurora	DCTCP
有效状态维度	11	流级统计	10	N/A
ECN 状态利用	是	否	否	是
CA 状态利用	是	否	否	否
决策机制	规则	规则	DNN	公式
学习作用对象	参数	参数	策略	无
自适应粒度	每流/每事件	流组/分钟级	每决策	无
决策位置	栈外进程	内核数据面	栈外/栈内	内核
训练需求	无	无（在线搜索）	小时/天	无
差异化响应	是	否	否	否
生产部署证据	无	有	无	有

vs. DCTCP [13]: Swift 直接读取 ECN 协议栈状态进行离散判断，而 DCTCP 用标记比例 α 做连续调整；Swift 按信号类型施加差异化保留因子（0.70/0.75/0.50）而非统一的 $1 - \alpha/2$ ；Swift 同时利用丢包、ECN 与 CA 状态三类信号，且参数在线自适应。需要注意的是，DCTCP 的比例响应在标记密集时更平滑，而 Swift 的离散响应依赖 D_{max} 与冻结机制来避免过度收缩。

vs. BBR [6]: Swift 综合多种显式拥塞信号，BBR 仅依赖速率与 RTT 探测。在洁净数据集中，BBR 于千兆级链路上仅取得 0.48~0.52 的利用率、在 leaf_spine_20g 上仅 0.37，而 Swift 在同样场景下达到饱和；在 UDP 突发下 BBR 的吞吐量保持率均值 39.07%、最差场景 9.69%，Jain 指数均值 0.844（最低 0.48），显著低于 Swift。需要说明的是，在两个微秒级 RTT 场景中 BBR 的吞吐量不足瓶颈的 1%，审计判定其为 ns-3.40 中 pacing 与 RTT 分辨率失配导致的实现性退化，已按规则 D 排除，因此本文不将该数据点用作 Swift 优于 BBR 的证据。反之，在 cross_dc_wan 与 wan_metro 等长 RTT 链路上 BBR 吞吐量高于 Swift，本文亦如实记录。

5.3 适用边界与未来方向

浅 BDP 链路上的排队延迟代价：Swift 以队列驻留换取零丢包，这一取舍在缓冲深度远大于 BDP 的低速率或长 RTT 接入链路上表现为明显的延迟抬升。相对更优的丢包类基线，纯 TCP 设置下的抬升幅度依次为：dc_100m +128.8%、wifi_n +104.5%、wifi_legacy +78.6%、lte_poor +78.4%、

nr_5g_edge +67.8%、dc_500m +60.6%、lte_good +59.6%，千兆级饱和链路（asymmetric_high、congested_heavy、symmetric_low）约 +26%，satellite_geo +7.2%。其机理是 $\alpha \times BDP$ 目标窗口在 α 上界 1.30 处仍高于 BDP，而这些场景的瓶颈队列受 $\max(\cdot, 100)$ 报文下限约束、相对 BDP 过深，于是稳态下形成可持续的队列驻留。可探索的改进方向包括：（1）引入显式排队时延估计 $q_{est} = RTT_{smoothed} - RTT_{min}$ 作为 α 调节的第四路输入，实现排队延迟感知的 α 调度；（2）当 RTT 膨胀持续超过最小 RTT 感知阈值时把 α 强制收敛至 1.0 以下；（3）用多时间尺度 BDP 估计分离传播时延与排队时延。

长 RTT 路径上的吞吐量不足：cross_dc_wan（1 Gbps、单向 5.02 ms）中 Swift 仅取得 709.72 Mbps，较 NewReno 低 28.9%、较 CUBIC 低 29.0%，是本文最主要的负向结果。归档产物使用的投递速率估计为逐 ACK 公式 $segmentsAched \cdot segSize / lastRtt$ ，其结果约等于真实带宽除以每 RTT 的 ACK 数量，因此在长 RTT 路径上系统性低估带宽，进而使 BDP 与 $\alpha \times BDP$ 目标窗口塌缩、窗口被 $200 \times MSS$ 安全下限钉住。v3.0.0 已将其替换为跨度不小于 $2 RTT_{min}$ 的滑动时间窗口估计（见第 4.1 节），预期可缓解该问题；但在完成全矩阵重跑之前，本文不宣称该修复的定量收益。此外，在 UDP 突发设置下 wan_metro 是 Swift 未取得最高吞吐量的唯一场景（801.06 Mbps，较 CUBIC 低 0.24%），同属该类边界。

参数预设依赖：保留因子（0.70/0.75/0.50）、 α 边界（[0.85, 1.30]）、连续递减阈值（ $D_{max} = 3$ ）与降窗后冻结的 ACK 事件数（4）均为手动调优的预设值。未来可通过元学习 [24] 或如 Gemini [19] 所示的带外贝叶斯优化实现参数的在线搜索，二者可与本文的每事件决策回路组合成分层结构。

深度 RL 增强：当前基于规则的策略可解释性强但表示能力有限。可探索离线强化学习训练神经网络策略，再通过模型蒸馏提取为规则，兼顾表示能力和实时性。

部署代价与真实环境验证：每个窗口调整事件一次跨进程往返是当前实现的主要部署障碍。后续需将

决策逻辑内联为 Linux 内核 TCP 模块或 QUIC 用户态拥塞控制插件，并在覆盖无线/蜂窝接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异（NIC offload、中断合并、NUMA 效应、终端操作系统调度与无线链路波动）可能显著影响实际性能。

证据边界说明：本文全部定量结论的唯一数据来源是 logs/summary/kpi_forward.csv，即仅统计前向数据流的重导出 KPI；两份旧绘图汇总表的吞吐量与延迟列因包含反向 ACK 流而被取代（规则 A）。跨协议对比仅在通过规则 B（排除汇聚端口不一致、混用两个 sim.cc 版本的 27 个组合）、规则 C（排除重复配置 dc_oversub_10to1 与错标场景 satellite_leo）与规则 D（在两个微秒级 RTT 场景中排除退化的 BBR 基线）筛选后的 19 个纯 TCP 场景与 15 个 UDP 突发场景上进行，全部排除记录见 logs/error.txt 的“AUDIT PASS 2026-08-20”条目。由此产生三条明确边界：其一，本文不给出任何 LEO 卫星链路结论，因唯一带该标签的场景实为 100/50 Mbps、5/20 ms 链路的错标副本，GEO 卫星结论不受影响；其二，上述延迟抬升与 cross_dc_wan 吞吐回落不进入头部结论，仅用于机制分析与参数敏感性讨论；其三，按规则 F，所有归档产物均生成于 v3.0.0 修复之前——当时瓶颈为不标记 CE 的 PfifoFast 队列且 ECN 仅对 Swift 开启，故 ECN 分支未被触发，投递速率仍使用逐 ACK 公式，奖励自适应仍使用固定阈值（见仓库 CHANGELOG.md）。因此本文各表数值刻画的是修复前实现在洁净场景集上的行为；v3.0.0 行为的定量评估须在完成同一实验矩阵重跑、异常筛查与结果溯源之后才能作为新结论引用。

6 结论

本文提出了 Swift——一种面向远距离传输与异构终端（手机、笔记本、台式机、边缘设备）的强化学习辅助多信号融合自适应拥塞控制算法。其核心技术贡献收敛为三点：（1）**多信号拥塞状态感知**，把 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节组织为 11 维有效观测子空间，并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包；（2）**强化学习辅助的自适应融合控制**，由最小 RTT 感知的 RTT 膨胀比、奖励相对慢速基线的偏移与连续增长趋势共同调节每流 $\alpha \in [0.85, 1.30]$ ，并以有界步长逼近由滑动时间窗口投递速率最大值滤波得到的 $\alpha \times BDP$ 目标窗口；（3）**面向不确定拥塞信号的稳定性与安全机制**，包括差异化保留因子（0.70/0.75/0.50）、连续递减下限 $D_{max} = 3$ 、降窗后 4 个 ACK 事件冻结、不奖励队列驻留的阈值更新、窗口钳位与超时丢弃旧动作。

在 ns-3.40 平台上，本文以仅统计前向数据流的重导出 KPI（logs/summary/kpi_forward.csv）为唯一数据来源，并在经四条排除规则审计后保留的 19 个纯 TCP 场景与 15 个 UDP 突发场景上进行评估。结果显示：在微秒级 RTT 高速链路上，Swift 在吞吐量与 NewReno/CUBIC 完全一致（相对差 +0.000%）的同时把平均单向延迟降低 82.1%~93.6%；在 nr_5g_embb、

wifi_ax、wan_metro、wifi_ac 等中速接入与城域链路上延迟降低 10.2%~41.0%；在 GEO 卫星链路上吞吐量相对 NewReno/CUBIC 均值提升 29.8%，UDP 突发性扩大至 87.8%；Swift 在两种设置的全部场景中丢包率严格为 0.00%（19/19 与 15/15），是四种协议中唯一做到这一点的算法，Jain 公平性指数恒为 1.00；UDP 突发性其吞吐量保持率均值 80.54%、最差场景 66.51%，优于 NewReno（75.86%/35.47%）、CUBIC（78.78%/53.75%）与 BBR（39.07%/9.69%），并在 15 个场景中的 14 个取得最高吞吐量。

本文同时如实披露混合与负向结果：cross_dc_wan 中 Swift 吞吐量较 NewReno/CUBIC 低约 29%，源于修复前逐 ACK 投递速率估计对长 RTT 路径带宽的系统性低估，v3.0.0 已改用时间窗口估计但尚未重跑验证；在缓冲深度远大于 BDP 的低速率与长 RTT 接入链路上，Swift 以队列驻留换取零丢包，延迟相对更优的丢包类基线抬升 7.2%~128.8%；UDP 突发性 wan_metro 的吞吐量较 CUBIC 低 0.24%。这些边界直接指向后续工作：排队延迟感知的 α 调度、多时间尺度 BDP 估计，以及把每事件决策回路内联到内核或用户态协议栈以消除跨进程开销。综合而言，本文的结论是：以协议栈内部信号驱动的可解释规则策略，可在不损失吞吐量的前提下同时取得零丢包与理想流间公平性，其代价是浅 BDP 链路上的排队延迟；该结论的适用范围以本文列出的洁净场景集与 v3.0.0 修复前的实现版本为界。

参考文献

- [1] Jacobson V. Congestion avoidance and control. ACM SIGCOMM CCR, 1988, 18(4): 314-329.
- [2] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP's fast recovery algorithm. RFC 6582, 2012.
- [3] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant. ACM SIGOPS OSR, 2008, 42(5): 64-74.
- [4] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet. Commun. ACM, 2012, 55(1): 57-65.
- [5] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet. IEEE JSAC, 1995, 13(8): 1465-1480.
- [6] Cardwell N, Cheng Y, et al. BBR: Congestion-based congestion control. ACM Queue, 2016, 14(5): 20-53.
- [7] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet. In Proc. NSDI, 2018: 329-342.
- [8] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control. In Proc. ICNP, 2017: 1-10.
- [9] Tan K, Song J, et al. A compound TCP approach for high-speed and long distance networks. In Proc. INFOCOM, 2006: 1-12.
- [10] Liu S, Başar T, Srikant R. TCP-Illinois: A loss-and delay-based congestion control algorithm. Perf. Eval., 2008, 65(6-7): 417-440.
- [11] Mascolo S, Casetti C, et al. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links. In Proc. MobiCom, 2001: 287-297.

- [12] Ramakrishnan K, Floyd S, Black D. The addition of ECN to IP. RFC 3168, 2001.
- [13] Alizadeh M, Greenberg A, et al. Data center TCP (DCTCP). ACM SIGCOMM CCR, 2010, 40(4): 63-74.
- [14] Li Y, Miao R, et al. HPCC: High precision congestion control. In Proc. SIGCOMM, 2019: 44-58.
- [15] Jay N, Rotman N, et al. A deep reinforcement learning perspective on Internet congestion control. In Proc. ICML, 2019: 3050-3059.
- [16] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control. ACM SIGCOMM CCR, 2013, 43(4): 123-134.
- [17] Jay N, Rotman N, et al. Internet congestion control via deep reinforcement learning. In Proc. NeurIPS, 2019.
- [18] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control. In Proc. SIGCOMM, 2020: 632-647.
- [19] Yang W, Liu Y, Tian C, Jiang J, Guo L. Gemini: Divide-and-conquer for practical learning-based Internet congestion control. In Proc. IEEE INFOCOM, 2023.
- [20] Dong M, Li Q, et al. PCC: Re-architecting congestion control for consistent high performance. In Proc. NSDI, 2015: 395-408.
- [21] Riley G F, Henderson T R. The ns-3 network simulator. Springer, 2010: 15-34.
- [22] Gawłowicz P, Zubow A. ns-3 meets OpenAI Gym: The playground for ML in networking research. In Proc. MSWiM, 2019: 113-120.
- [23] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination. DEC Research Report, 1984.
- [24] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks. In Proc. ICML, 2017: 1126-1135.